Project Setup

- **Why Virtual Env-**
    - **Isolation**: Keeps dependencies separate for each project to avoid conflicts.
    - **Clean environment**: Prevents cluttering the global Python installation.
      Reproducibility: Makes it easy to recreate the same environment using requirements.txt.
    - **Security**: Limits the risk by not installing unnecessary packages globally.
    - **Python version control**: Allows using different Python versions for different projects (with tools like pyenv).

- Venv setup
    - Mac
        - python3 -m venv .venv
        - source ./venv/bin/activate
    - Windows
        - python -m venv venv
        - venv\Scripts\activate
- Create Folder rag
- requirements.txt

```
python-dotenv
langgraph
langchain
langchain-core
langchain-openai
langchain-huggingface
langchain-text-splitters
langchain-chroma
chromadb
sentence-transformers
```

- pip install -r requirements.txt
- .env file

```
OPENAI_API_KEY
```

- GitHub Repo (all code including sample project) - https://github.com/keertipurswani/AgenticAISystemDesign

Basic Semantic RAG

- OpenAI API Key - https://platform.openai.com/settings/organization/api-keys
- LangChain Docs - https://docs.langchain.com/oss/python/langchain/overview
    - https://docs.langchain.com/oss/python/langchain/rag
    - https://docs.langchain.com/oss/python/langchain/retrieval
- Setup Groq Key
    - https://groq.com/
    - Create key - https://console.groq.com/keys
    - https://console.groq.com/docs/models
- 1_semanticrag.py

```
import argparse
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

from langchain_core.documents import Document
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_openai import ChatOpenAI
from langchain_core.tools.retriever import create_retriever_tool
from langchain.agents import create_agent

CHUNK_SIZE = 256

def load_codebase(repo_path: str) -> list:
    docs = []
    for path in Path(repo_path).rglob("*.py"):
        text = path.read_text(encoding="utf-8", errors="ignore")
        docs.append(Document(page_content=text, metadata={"source": str(path)}))
    return docs


def chunk_code(docs: list) -> list:
    splitter = RecursiveCharacterTextSplitter.from_language(
        language=Language.PYTHON,
        chunk_size=CHUNK_SIZE,
        chunk_overlap=32,
    )
    return splitter.split_documents(docs)
```

```
def build_vector_store(chunks: list) -> Chroma:
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    return Chroma.from_documents(chunks, embedding=embeddings)


def build_agent(vector_store: Chroma):
    retriever_tool = create_retriever_tool(
        vector_store.as_retriever(search_kwargs={"k": 4}),
        name="search_codebase",
        description="Search the codebase for relevant functions, classes, or
logic.",
    )
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return create_agent(
        llm, tools=[retriever_tool],
        system_prompt=(
            "You are a senior engineer. Always use search_codebase before answering.
"
            "Reference specific file and function names. "
            "If not found say 'I could not find that in the codebase'."
        ),
    )


if __name__ == "__main__":

    parser = argparse.ArgumentParser()
    parser.add_argument("--repo", default=str(Path(__file__).parent.parent /
"sample_project"))
    args = parser.parse_args()
    repo_path = str(Path(args.repo).resolve())


    docs = load_codebase(repo_path)
    chunks = chunk_code(docs)
    print(f"Loaded {len(docs)} files → {len(chunks)} chunks
(chunk_size={CHUNK_SIZE})")


    vector_store = build_vector_store(chunks)
    agent = build_agent(vector_store)


    print("Ready. Ask your question. Type 'exit' to quit")
```

```
while True:
    question = input("\nYou: ").strip()
    if not question or question.lower() in ("exit", "quit"):
        break

    for step in agent.stream(
        {"messages": [{"role": "user", "content": question}]},
        stream_mode="values",
    ):
        last_msg = step["messages"][-1]
        if not getattr(last_msg, "tool_calls", None):
            print(f"Agent: {last_msg.content}")
```

Change Chunk Size and notice if some queries can be answered -

```
CHUNK_SIZE = 1500
```

RecursiveCharacterTextSplitter (Python)

- Treats code as text and uses Python-specific patterns (class, def, blank lines, etc.) to find better chunk boundaries.
- Tries larger code boundaries first and progressively falls back to smaller ones until the chunk size is satisfied.
- Understands Python-like text patterns, but does not understand actual code structure or relationships.
- Better than pure character chunking, but less accurate than AST-based chunking.

Try splitting by "class"
↓
Still too big?
↓
Try splitting by "def"
↓
Still too big?
↓
Try splitting by blank lines
↓
Still too big?
↓
Try splitting by newlines
↓
Still too big?
↓
Split by characters

Tracing Queries - LangSmith

Docs - https://docs.langchain.com/langsmith/home

- .env

```
LANGSMITH_TRACING="true"
LANGSMITH_API_KEY=...
```

Limit Model and Tool Calls in LangChain

- https://reference.langchain.com/python/langchain/agents/middleware/model_call_limit/ModelCallLimitMiddleware
- https://reference.langchain.com/python/langchain/agents/middleware/tool_call_limit/ToolCallLimitMiddleware

```
from langchain.agents.middleware import ModelCallLimitMiddleware
from langchain.agents.middleware import ToolCallLimitMiddleware
```

```
def build_agent(vector_store: Chroma):
    retriever_tool = create_retriever_tool(
        vector_store.as_retriever(search_kwargs={"k": 4}),
        name="search_codebase",
        description="Search the codebase for relevant functions, classes, or
logic.",
    )
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return create_agent(
        llm, tools=[retriever_tool],
        system_prompt=(
            "You are a senior engineer. Always use search_codebase before answering.
"
            "Reference specific file and function names. "
            "If not found say 'I could not find that in the codebase'."
        ),
        middleware=[
            ModelCallLimitMiddleware(run_limit=5, exit_behavior="end"),
            ToolCallLimitMiddleware(tool_name="search_codebase", run_limit=2,
exit_behavior="end")
        ]
    )
```

Semantic RAG with AST

Why AST?

- Source code is more than just text; it has structure.
- AST (Abstract Syntax Tree) converts code into a tree representation that captures its meaning.
- It helps us identify logical code units such as classes, functions, loops, and conditionals.
- For Code RAG, AST allows us to chunk code based on structure rather than arbitrary character counts.
- This preserves context and improves retrieval quality.

Where else are ASTs used?

- Compilers and interpreters to understand and execute code.
- Linters and static analysis tools to detect issues and enforce coding standards.
- IDEs for autocomplete, refactoring, and code navigation.

AST APIs Used

ast.parse()

- Converts source code into an AST tree.

ast.dump()

- Prints a human-readable representation of the AST.
- Useful for visualizing and understanding the tree structure.

ast.walk()

- Traverses all nodes in the tree.

ast.iter_child_nodes()

- Returns the direct children of a node.
- Used to establish parent-child relationships.

ast.get_source_segment()

- Extracts the original source code corresponding to a specific AST node.
- Used to create chunks for embedding and retrieval.

AST vs RecursiveCharacterTextSplitter

RecursiveCharacterTextSplitter

- Treats code as text and uses Python-specific patterns (class, def, newlines) to find chunk boundaries.
- Does not understand actual code structure or relationships.

AST Chunking

- Parses code into a syntax tree before chunking.
- Understands classes, functions, methods, and their relationships.

Key Difference: AST understands the code first and then chunks it.

AST-Based Chunking Flow

Source Code → AST Tree → Find Classes/Functions → Extract Source Code → Create Chunks → Embed & Store

AST Visualization (Optional)

2_ast_demo.py

```
import ast
import textwrap

# ---------------------------------------------------------------------------
# Helper — pretty-print an AST node tree with indentation
# ---------------------------------------------------------------------------
def show_tree(code: str) -> None:

   print("\n--- SOURCE ---")
   print(code.strip())

   print("\n--- AST dump ---")
   tree = ast.parse(code)
   print(ast.dump(tree, indent=2))

# ---------------------------------------------------------------------------
# EXAMPLE 1  — a simple expression
# ---------------------------------------------------------------------------

# Key observations Python produces for `x = 1 + 2`:
#
#   Module
#    └── Assign
#        ├── targets: [Name(id='x')]
#        └── value:   BinOp
#                     ├── left:  Constant(value=1)
#                     ├── op:    Add()
#                     └── right: Constant(value=2)

show_tree("x = 1 + 2")
```

```
# ------------------------------------------------------------------------
# EXAMPLE 2  — a function definition
# ------------------------------------------------------------------------

# The tree shows:
#   FunctionDef
#   ├── name: 'greet'
#   ├── args: arguments  (name, annotation=str)
#   ├── returns: Name(id='str')
#   └── body:
#       └── Return
#           └── BinOp(Constant("Hello, ") + Name(id='name'))

show_tree(textwrap.dedent("""\
   def greet(name: str) -> str:
       return "Hello, " + name
"""))


# ------------------------------------------------------------------------
# EXAMPLE 3  — a class with methods
# ------------------------------------------------------------------------

show_tree(textwrap.dedent("""\
   class Dog:
       def __init__(self, name):
           self.name = name

       def bark(self):
           return f"Woof! I am {self.name}"
"""))
```

Example Code

3_semanticrag_ast.py

```
import ast
import argparse
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

from langchain_core.documents import Document
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_openai import ChatOpenAI
from langchain_core.tools.retriever import create_retriever_tool
from langchain.agents import create_agent



# ------------------------------------------------------------------------
# 1. LOAD
# ------------------------------------------------------------------------

def load_codebase(repo_path: str) -> list:
    docs = []
    for path in Path(repo_path).rglob("*.py"):
        text = path.read_text(encoding="utf-8", errors="ignore")
        docs.append(Document(page_content=text, metadata={"source": str(path)}))
    return docs



# ------------------------------------------------------------------------
# 2. CHUNK — AST-based (one chunk per class or module-level function)
# ------------------------------------------------------------------------

def chunk_code(docs: list) -> list:
    chunks = []

    for doc in docs:
        source = doc.metadata.get("source", "unknown")
        code = doc.page_content

        try:
            tree = ast.parse(code)
```

```
        except SyntaxError:
            chunks.append(doc)
            continue

        # Add parent references — ast.walk doesn't expose parents by default
        for node in ast.walk(tree):
            for child in ast.iter_child_nodes(node):
                child.parent = node

        for node in ast.walk(tree):
            parent = getattr(node, "parent", None)

            # Top-level class → grab whole class including all methods
            if isinstance(node, ast.ClassDef) and isinstance(parent, ast.Module):
                text = ast.get_source_segment(code, node)
                if text:
                    chunks.append(Document(
                        page_content=text,
                        metadata={"source": source, "type": "class", "name":
node.name},
                    ))

            # Module-level function — not a method inside a class
            elif isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)) and
isinstance(parent, ast.Module):
                text = ast.get_source_segment(code, node)
                if text:
                    chunks.append(Document(
                        page_content=text,
                        metadata={"source": source, "type": "function", "name":
node.name},
                    ))

    return chunks


# ---------------------------------------------------------------------------
# 3. EMBED & STORE
# ---------------------------------------------------------------------------

def build_vector_store(chunks: list) -> Chroma:
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    return Chroma.from_documents(chunks, embedding=embeddings)
```

```python
# ---------------------------------------------------------------------------
# 4. RETRIEVER + AGENT
# ---------------------------------------------------------------------------

def build_agent(vector_store: Chroma):
    retriever_tool = create_retriever_tool(
        vector_store.as_retriever(search_kwargs={"k": 4}),
        name="search_codebase",
        description="Search the codebase for relevant functions, classes, or
logic.",
    )
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return create_agent(
        llm, tools=[retriever_tool],
        system_prompt=(
            "You are a senior engineer. Always use search_codebase before answering.
"
            "Reference specific file and function names. "
            "If not found say 'I could not find that in the codebase'."
        ),
    )



# ---------------------------------------------------------------------------
# 5. MAIN
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--repo", default=str(Path(__file__).parent.parent /
"sample_project"))
    args = parser.parse_args()
    repo_path = str(Path(args.repo).resolve())

    docs = load_codebase(repo_path)
    chunks = chunk_code(docs)
    print(f"Loaded {len(docs)} files → {len(chunks)} chunks (AST-based)")

    vector_store = build_vector_store(chunks)
    agent = build_agent(vector_store)

    print("Ready. Ask your question. Type 'exit' to quit")
    while True:
        question = input("\nYou: ").strip()
        if not question or question.lower() in ("exit", "quit"):
```

```
        break

    for step in agent.stream(
        {"messages": [{"role": "user", "content": question}]},
        stream_mode="values",
    ):
        last_msg = step["messages"][-1]
        if not getattr(last_msg, "tool_calls", None):
            print(f"Agent: {last_msg.content}")
```

Interesting Queries to try (Try changing k to 1) -

- What is AuthService for?
- What are the functions inside AuthService?
- If I call complete_task, what will happen if the session token is expired — trace the full call chain and list every method involved across all files.
- If a user's account is deactivated (is_active = False), which specific service methods in the codebase would they be blocked from calling?
- What protects against privilege escalation in this codebase?
- Trace the complete call chain when create_task is called — list every method invoked across all files in order.

Queries not working well-

- Find every place in the codebase where BillingError is raised.
- Which methods in the codebase check owner.is_active before proceeding?
- Find every place in the codebase where AuditEventType.PERMISSION_DENIED is recorded.
- Which file imports both Task and Status?
- Where is _user_plans defined and what stores it?

Lexical RAG

- Retrieves documents using keyword matching.
- Uses BM25 instead of embeddings.
- Scores documents using term frequency and rarity of terms.
- Excellent for exact identifiers, class names, APIs, and error messages.
- Does not understand semantic meaning.

BM25 (Best Matching 25)

BM25 = Inverted Index + Ranking Algorithm

- Lexical (keyword-based) retrieval algorithm used by default in search engines like Elasticsearch and Apache Solr.
- Builds an **inverted index** (word → documents containing the word) for fast retrieval.
- Retrieves candidate documents using keyword matches and then ranks them using the BM25 scoring algorithm.
- Scores are influenced by term frequency, word rarity (IDF), and document length.
- Does not understand meaning or synonyms - only matching words.

BM25 Visualization (Optional)

```
import math
import re
from collections import defaultdict

DOCS = {
   "D0": "create_invoice generate invoice for order calculate invoice total",
   "D1": "create_payment process payment validate payment method charge card",
   "D2": "create_statement generate statement for customer calculate amount due",
   "D3": "send_invoice email invoice to customer attach invoice pdf",
   "D4": "process_refund reverse payment refund amount to customer card",
}

DOC_IDS   = list(DOCS.keys())
DOC_TEXTS = list(DOCS.values())

STOP_WORDS = {"a", "an", "the", "and", "or", "to", "in", "on", "for",
              "of", "with", "is", "are", "was", "be", "at", "by"}

def tokenize(text: str) -> list:
   return [t for t in re.findall(r"[a-z0-9_]+", text.lower()) if t not in
STOP_WORDS]
```

```
# ---------------------------------------------------------------------------
# STEP 1 — Inverted Index
# ---------------------------------------------------------------------------

print("--- STEP 1: Inverted Index ---")
print()
print("  Forward index  : document → words it contains   (what you read normally)")
print("  Inverted index : word     → documents containing it  (what search needs)")
print()
print("  Given a query word, the inverted index tells you instantly which
documents")
print("  to look at — without scanning every document in the corpus.")
print()

tokenized      = [tokenize(text) for text in DOC_TEXTS]
inverted_index = defaultdict(dict)
doc_lengths    = {}

for doc_id, tokens in zip(DOC_IDS, tokenized):
    doc_lengths[doc_id] = len(tokens)
    for token in tokens:
        inverted_index[token][doc_id] = inverted_index[token].get(doc_id, 0) + 1

# Grid: only terms that appear in more than one doc
shared_terms = sorted(
    t for t, postings in inverted_index.items() if len(postings) > 1
)

col_w = 6
print(f"  {'term':<18}", end="")
for doc_id in DOC_IDS:
    print(f"{doc_id:>{col_w}}", end="")
print()
print(f"  {'-'*18}", end="")
for _ in DOC_IDS:
    print(f"{'-'*col_w}", end="")
print()

for term in shared_terms:
    print(f"  {term:<18}", end="")
    for doc_id in DOC_IDS:
        tf   = inverted_index[term].get(doc_id, 0)
        cell = str(tf) if tf else "."
        print(f"{cell:>{col_w}}", end="")
    print()
```

```
print()
print("  Number = how many times the word appears in that document  (term
frequency)")
print("  Dot    = word is absent → BM25 skips this document for that term
entirely")


# ------------------------------------------------------------------------
# STEP 2 — The three factors BM25 uses to score
# ------------------------------------------------------------------------

print()
print("--- STEP 2: How BM25 scores a document ---")
print()
print("  BM25 score  =  Term Frequency  +  Word Rarity  -  Length Penalty")
print()
print("  Factor 1 — Term Frequency (TF)")
print("    More occurrences of the query word in a document → higher score")
print("    But each extra occurrence adds less than the last (diminishing
returns)")
print()
print("  Factor 2 — Word Rarity  (IDF — Inverse Document Frequency)")
print("    Word appears in every document → low IDF, nearly useless for ranking")
print("    Word appears in one document   → high IDF, strongly discriminating")
print()
print("  Factor 3 — Document Length Penalty")
print("    A long document contains more words just by size, not relevance")
print("    BM25 slightly penalises documents longer than average")


# ------------------------------------------------------------------------
# STEP 3 — IDF table: see word rarity in action
# ------------------------------------------------------------------------

N     = len(DOC_IDS)
avgdl = sum(doc_lengths.values()) / N
k1    = 1.5
b     = 0.75

def idf(term: str) -> float:
   df = len(inverted_index[term])
   return math.log((N - df + 0.5) / (df + 0.5) + 1)

print()
```

```
print("--- STEP 3: IDF — word rarity scores ---")
print()
print(f"  {'term':<18}  {'in how many docs':>18}  {'IDF score':>10}  note")
print(f"  {'-'*18}  {'-'*18}  {'-'*10}  {'-'*30}")
for term in shared_terms:
    df       = len(inverted_index[term])
    score    = idf(term)
    note     = "← common, less useful" if df >= 3 else ("← rare, more useful" if df
== 1 else "")
    print(f"  {term:<18}  {df:>18}  {score:>10.2f}  {note}")



# ---------------------------------------------------------------------------
# STEP 4 — Ranking: query that BM25 handles well
# ---------------------------------------------------------------------------

def bm25_score(query_tokens: list, doc_id: str) -> float:
    dl    = doc_lengths[doc_id]
    score = 0.0
    for term in query_tokens:
        tf = inverted_index[term].get(doc_id, 0)
        if tf == 0:
            continue
        tf_norm = (tf * (k1 + 1)) / (tf + k1 * (1 - b + b * dl / avgdl))
        score  += idf(term) * tf_norm
    return score

def show_ranking(query: str) -> None:
    tokens = tokenize(query)
    scores = sorted(
        [(doc_id, bm25_score(tokens, doc_id)) for doc_id in DOC_IDS],
        key=lambda x: x[1], reverse=True,
    )
    print(f"  Query : \"{query}\"")
    print(f"  Tokens: {tokens}")
    print()
    for rank, (doc_id, score) in enumerate(scores, 1):
        bar  = "" * int(score * 8)
        mark = " ✓" if score > 0 else ""
        print(f"  rank {rank}  {doc_id}  score={score:.2f}  {bar:<20}{mark}")
        print(f"           \"{DOCS[doc_id]}\"")



print()
print("--- STEP 4: BM25 ranking — query it handles well ---")
```

```
print()
show_ranking("create invoice")

print()
print("  BM25 works perfectly here.")
print("  D0 and D3 rank highest because they both contain 'invoice' multiple
times.")
print("  D1/D2/D4 score low or zero — they don't share the exact words.")



# ------------------------------------------------------------------------
# STEP 5 — Where BM25 fails: the lexical gap
# ------------------------------------------------------------------------

print()
print("--- STEP 5: Where BM25 fails — the lexical gap ---")
print()
show_ranking("generate bill")

print()
print("  D0 sneaks in at rank 1 — but only because it literally contains
'generate'.")
print("  BM25 did NOT understand that 'bill' = 'invoice'.")
print()
print("  D3 (send_invoice) scores zero — it's clearly about invoices but has no")
print("  exact match for 'generate' or 'bill', so BM25 ignores it entirely.")
print()
print("  Why?")
print("    Query says  'generate'  →  documents say  'create'")
print("    Query says  'bill'      →  documents say  'invoice'")
print()
print("  BM25 only matches exact words. It has no concept of meaning.")
print("  'generate' and 'create' mean the same thing — BM25 doesn't know that.")
print()
print("  This is the lexical gap, and it's exactly why semantic/vector search
exists.")
print("  Semantic search embeds both query and document into vector space,")
print("  so 'generate bill' lands close to 'create invoice' even without shared
words.")
```

Example Code

```
import argparse
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

from langchain_core.documents import Document
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter
from langchain_core.retrievers import BaseRetriever
from langchain_core.callbacks import CallbackManagerForRetrieverRun
from rank_bm25 import BM25Okapi
from langchain_openai import ChatOpenAI
from langchain_core.tools.retriever import create_retriever_tool
from langchain.agents import create_agent



# ---------------------------------------------------------------------------
# 1. LOAD
# ---------------------------------------------------------------------------

CHUNK_SIZE = 256

def load_codebase(repo_path: str) -> list:
    docs = []
    for path in Path(repo_path).rglob("*.py"):
        text = path.read_text(encoding="utf-8", errors="ignore")
        docs.append(Document(page_content=text, metadata={"source": str(path)}))
    return docs



# ---------------------------------------------------------------------------
# 2. CHUNK — same character-based splits as demo 1
# ---------------------------------------------------------------------------

def chunk_code(docs: list) -> list:
    splitter = RecursiveCharacterTextSplitter.from_language(
        language=Language.PYTHON,
        chunk_size=CHUNK_SIZE,
        chunk_overlap=32,
    )
    return splitter.split_documents(docs)
```

```
# ---------------------------------------------------------------------------
# 3. BM25 INDEX — no embeddings, pure term frequency
# ---------------------------------------------------------------------------

class BM25Retriever(BaseRetriever):
    """Thin LangChain-compatible retriever backed by rank_bm25."""
    docs: list
    bm25: object
    k: int = 4

    class Config:
        arbitrary_types_allowed = True

    def _get_relevant_documents(
        self, query: str, *, run_manager: CallbackManagerForRetrieverRun
    ) -> list:
        tokens = query.lower().split()
        scores = self.bm25.get_scores(tokens)
        top_indices = sorted(range(len(scores)), key=lambda i: scores[i],
reverse=True)[: self.k]
        return [self.docs[i] for i in top_indices]


def build_bm25_retriever(chunks: list) -> BM25Retriever:
    tokenized = [doc.page_content.lower().split() for doc in chunks]
    bm25 = BM25Okapi(tokenized)
    return BM25Retriever(docs=chunks, bm25=bm25)


# ---------------------------------------------------------------------------
# 4. RETRIEVER + AGENT
# ---------------------------------------------------------------------------

def build_agent(retriever: BaseRetriever):
    retriever_tool = create_retriever_tool(
        retriever,
        name="search_codebase",
        description="Search the codebase for relevant functions, classes, or
logic.",
    )
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return create_agent(
        llm, tools=[retriever_tool],
        system_prompt=(
```

```
            "You are a senior engineer. Always use search_codebase before answering.
"
            "Reference specific file and function names. "
            "If not found say 'I could not find that in the codebase'."
        ),
    )


# ---------------------------------------------------------------------------
# 5. MAIN
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--repo", default=str(Path(__file__).parent.parent /
"sample_project"))
    args = parser.parse_args()
    repo_path = str(Path(args.repo).resolve())



    docs = load_codebase(repo_path)
    chunks = chunk_code(docs)
    print(f"Loaded {len(docs)} files → {len(chunks)} chunks
(chunk_size={CHUNK_SIZE})")


    retriever = build_bm25_retriever(chunks)
    agent = build_agent(retriever)


    print("Ready. Ask your question. Type 'exit' to quit")
    while True:
        question = input("\nYou: ").strip()
        if not question or question.lower() in ("exit", "quit"):
            break

        for step in agent.stream(
            {"messages": [{"role": "user", "content": question}]},
            stream_mode="values",
        ):
            last_msg = step["messages"][-1]
            if not getattr(last_msg, "tool_calls", None):
                print(f"Agent: {last_msg.content}")
```

Hybrid RAG

It is not a specific retrieval technique. It is a retrieval architecture that combines multiple retrieval strategies

- Combines results from multiple retrieval strategies.
- Each retriever contributes different strengths.
- Retrieved results are merged and ranked to produce the final context.

Hybrid retrieval can combine any retrieval approaches, for example:

- Lexical Search + Semantic Search
- Semantic Search + Graph Search
- Semantic Search + SQL Retrieval
- Multiple Vector Retrievers with different embedding models

```
import argparse
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

from langchain_core.documents import Document
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter
from langchain_core.retrievers import BaseRetriever
from langchain_core.callbacks import CallbackManagerForRetrieverRun
from pydantic import ConfigDict
from langchain_classic.retrievers.ensemble import EnsembleRetriever
from rank_bm25 import BM25Okapi
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_openai import ChatOpenAI
from langchain_core.tools.retriever import create_retriever_tool
from langchain.agents import create_agent

from langchain.agents.middleware import ModelCallLimitMiddleware
from langchain.agents.middleware import ToolCallLimitMiddleware

# ---------------------------------------------------------------------------
# 1. LOAD
# ---------------------------------------------------------------------------

CHUNK_SIZE = 256
```

```python
def load_codebase(repo_path: str) -> list:
    docs = []
    for path in Path(repo_path).rglob("*.py"):
        text = path.read_text(encoding="utf-8", errors="ignore")
        docs.append(Document(page_content=text, metadata={"source": str(path)}))
    return docs


# ---------------------------------------------------------------------------
# 2. CHUNK — character-based splits shared by both retrievers
# ---------------------------------------------------------------------------

def chunk_code(docs: list) -> list:
    splitter = RecursiveCharacterTextSplitter.from_language(
        language=Language.PYTHON,
        chunk_size=CHUNK_SIZE,
        chunk_overlap=32,
    )
    return splitter.split_documents(docs)


# ---------------------------------------------------------------------------
# 3a. SEMANTIC — dense vector retriever (same as demo 1/2)
# ---------------------------------------------------------------------------

def build_vector_retriever(chunks: list):
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    vector_store = Chroma.from_documents(chunks, embedding=embeddings)
    return vector_store.as_retriever(search_kwargs={"k": 4})


# ---------------------------------------------------------------------------
# 3b. LEXICAL — sparse BM25 retriever (same as demo 7)
# ---------------------------------------------------------------------------

class BM25Retriever(BaseRetriever):
    """Thin LangChain-compatible retriever backed by rank_bm25."""
    docs: list
    bm25: object
    k: int = 4

    model_config = ConfigDict(arbitrary_types_allowed=True)

    def _get_relevant_documents(
        self, query: str, *, run_manager: CallbackManagerForRetrieverRun
```

```
    ) -> list:
        tokens = query.lower().split()
        scores = self.bm25.get_scores(tokens)
        top_indices = sorted(range(len(scores)), key=lambda i: scores[i],
reverse=True)[: self.k]
        return [self.docs[i] for i in top_indices]


def build_bm25_retriever(chunks: list) -> BM25Retriever:
    tokenized = [doc.page_content.lower().split() for doc in chunks]
    bm25 = BM25Okapi(tokenized)
    return BM25Retriever(docs=chunks, bm25=bm25)


# ---------------------------------------------------------------------------
# 3c. HYBRID — fuse both via Reciprocal Rank Fusion (weights sum to 1.0)
# ---------------------------------------------------------------------------

def build_hybrid_retriever(chunks: list):
    """
    EnsembleRetriever merges results from both retrievers using RRF.
    weights=[0.5, 0.5] gives equal say to semantic and lexical signals.
    Raise the first weight for queries that need conceptual understanding;
    raise the second for queries with exact identifiers or error strings.
    """
    vector_retriever = build_vector_retriever(chunks)
    bm25_retriever = build_bm25_retriever(chunks)
    return EnsembleRetriever(
        retrievers=[vector_retriever, bm25_retriever],
        weights=[0.5, 0.5],
    )


# ---------------------------------------------------------------------------
# 4. AGENT
# ---------------------------------------------------------------------------

def build_agent(retriever):
    retriever_tool = create_retriever_tool(
        retriever,
        name="search_codebase",
        description="Search the codebase for relevant functions, classes, or
logic.",
    )
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

```
    return create_agent(
        llm, tools=[retriever_tool],
        system_prompt=(
            "You are a senior engineer. Always use search_codebase before answering.
"
            "Reference specific file and function names. "
            "If not found say 'I could not find that in the codebase'."
        ),
        middleware=[
            ModelCallLimitMiddleware(run_limit=5, exit_behavior="end"),
            ToolCallLimitMiddleware(tool_name="retriever_tool", run_limit=2,
exit_behavior="end")
        ]
    )



# ---------------------------------------------------------------------------
# 5. MAIN
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--repo", default=str(Path(__file__).parent.parent /
"sample_project"))
    parser.add_argument(
        "--weights", nargs=2, type=float, default=[0.5, 0.5],
        metavar=("SEMANTIC", "LEXICAL"),
        help="Retriever blend weights (must sum to 1.0). Default: 0.5 0.5",
    )
    args = parser.parse_args()
    repo_path = str(Path(args.repo).resolve())



    docs = load_codebase(repo_path)
    chunks = chunk_code(docs)
    print(f"Loaded {len(docs)} files → {len(chunks)} chunks
(chunk_size={CHUNK_SIZE})")
    print(f"Hybrid weights — semantic: {args.weights[0]}, lexical:
{args.weights[1]}")



    retriever = build_hybrid_retriever(chunks)
```

```
retriever.weights = args.weights  # allow CLI override
agent = build_agent(retriever)



print("Ready. Ask your question. Type 'exit' to quit")
while True:
    question = input("\nYou: ").strip()
    if not question or question.lower() in ("exit", "quit"):
        break

    for step in agent.stream(
        {"messages": [{"role": "user", "content": question}]},
        stream_mode="values",
    ):
        last_msg = step["messages"][-1]
        if not getattr(last_msg, "tool_calls", None):
            print(f"Agent: {last_msg.content}")
```

RAG Metrics

```
from dotenv import load_dotenv
from langchain_core.documents import Document
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_openai import OpenAIEmbeddings
import math

load_dotenv()

# ── Knowledge base ──────────────────────────────────────────────────
corpus = [
    Document(page_content="Full refund available within 30 days of purchase date.",
metadata={"id": "doc_1"}),
    Document(page_content="Digital products are refundable only if defective or
non-functional.",     metadata={"id": "doc_2"}),
    Document(page_content="Refund processing takes 5 to 7 business days after
approval.",              metadata={"id": "doc_3"}),
    Document(page_content="Submit refund requests through the support portal at
support.acme.com.",    metadata={"id": "doc_4"}),
    Document(page_content="Express shipping takes 2 business days. Standard shipping
takes 5-7 days.",   metadata={"id": "doc_5"}),
    Document(page_content="Free standard shipping on all orders over $50.",
metadata={"id": "doc_6"}),
    Document(page_content="Password must be 12 characters minimum with uppercase,
number, special char.", metadata={"id": "doc_7"}),
    Document(page_content="Passwords expire every 90 days. Cannot reuse last 5
passwords.",            metadata={"id": "doc_8"}),
    Document(page_content="The company was founded in 2010 and serves over 1 million
customers.",       metadata={"id": "doc_9"}),  # distractor — never relevant
    Document(page_content="API rate limit: 500 requests per minute for the
/v2/events endpoint.",       metadata={"id": "doc_10"}),
]

# ── Golden dataset ──────────────────────────────────────────────────
# Each entry: query + the doc IDs that are TRULY relevant to that query
golden_set = [
    {"query": "How long do I have to request a refund?",
     "relevant_ids": ["doc_1", "doc_2"]},   # both docs needed for a complete answer
    {"query": "How do I submit a refund request?",
     "relevant_ids": ["doc_4"]},
    {"query": "How long does refund processing take?",
     "relevant_ids": ["doc_3"]},
    {"query": "What are the shipping options and costs?",
```

```
    "relevant_ids": ["doc_5", "doc_6"]},
   {"query": "What are the password requirements?",
    "relevant_ids": ["doc_7", "doc_8"]},
   {"query": "What is the API rate limit?",
    "relevant_ids": ["doc_10"]},
]

# ── Build vector store ──────────────────────────────────────────────
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vs = InMemoryVectorStore.from_documents(corpus, embeddings)

# ── Metric functions ────────────────────────────────────────────────

def recall_at_k(retrieved_ids: list, relevant_ids: list, k: int) -> float:
   top_k = set(retrieved_ids[:k])
   relevant = set(relevant_ids)
   return len(top_k & relevant) / len(relevant) if relevant else 0.0

def precision_at_k(retrieved_ids: list, relevant_ids: list, k: int) -> float:
   top_k = retrieved_ids[:k]
   relevant = set(relevant_ids)
   return sum(1 for d in top_k if d in relevant) / k if k > 0 else 0.0

def mrr(retrieved_ids: list, relevant_ids: list) -> float:
   relevant = set(relevant_ids)
   for rank, doc_id in enumerate(retrieved_ids, start=1):
       if doc_id in relevant:
           return 1.0 / rank
   return 0.0

def hit_rate_at_k(retrieved_ids: list, relevant_ids: list, k: int) -> float:
   top_k = set(retrieved_ids[:k])
   return 1.0 if set(relevant_ids) & top_k else 0.0

def ndcg_at_k(retrieved_ids: list, relevant_ids: list, k: int) -> float:
   relevant = set(relevant_ids)
   dcg = sum(
       1.0 / math.log2(rank + 1)
       for rank, doc_id in enumerate(retrieved_ids[:k], start=1)
       if doc_id in relevant
   )
   ideal_hits = min(len(relevant), k)
   idcg = sum(1.0 / math.log2(rank + 1) for rank in range(1, ideal_hits + 1))
   return dcg / idcg if idcg > 0 else 0.0
```

```python
# ── Run evaluation across multiple K values ──────────────────────────────
results_by_k = {}

for K in [1, 3, 5]:
    results = []
    print(f"\n{'─'*80}")
    print(f"  K = {K}")
    print(f"{'─'*80}")
    print(f"  {'Query':48s} {'R@K':6s} {'P@K':6s} {'MRR':6s} {'Hit':6s} {'nDCG':6s}")
    print(f"  {'-'*80}")



    for item in golden_set:
        retrieved_docs = vs.similarity_search(item["query"], k=K)
        retrieved_ids  = [doc.metadata["id"] for doc in retrieved_docs]


        r  = recall_at_k(retrieved_ids, item["relevant_ids"], K)
        p  = precision_at_k(retrieved_ids, item["relevant_ids"], K)
        m  = mrr(retrieved_ids, item["relevant_ids"])
        hr = hit_rate_at_k(retrieved_ids, item["relevant_ids"], K)
        nd = ndcg_at_k(retrieved_ids, item["relevant_ids"], K)


        results.append({"query": item["query"], "relevant_ids": item["relevant_ids"],
                        "retrieved_ids": retrieved_ids,
                        "recall": r, "precision": p, "mrr": m, "hit_rate": hr, "ndcg": nd})

        print(f"  {item['query'][:48]:48s} {r:.2f}   {p:.2f}   {m:.2f}   {hr:.2f}   {nd:.2f}")


    print(f"  {'─'*80}")
    for metric in ["recall", "precision", "mrr", "hit_rate", "ndcg"]:
        avg = sum(r[metric] for r in results) / len(results)
        label = "✅" if avg >= 0.7 else ("⚠️ " if avg >= 0.5 else "❌")
        print(f"    {label} Mean {metric:12s}: {avg:.4f}")

    results_by_k[K] = results
```

```python
# ── Failure analysis (K=3) ──────────────────────────────────────────
print(f"\n{'─'*80}")
print("  Failure analysis  (K=3, recall < 1.0)")
print(f"{'─'*80}")
failures = [r for r in results_by_k[3] if r["recall"] < 1.0]
if failures:
    for r in failures:
        missed = set(r["relevant_ids"]) - set(r["retrieved_ids"])
        print(f"  Query    : {r['query']}")
        print(f"  Retrieved: {r['retrieved_ids']}")
        print(f"  Missed   : {sorted(missed)}")
        print()
else:
    print("  No failures — all queries achieved recall = 1.0 at K=3")
```

GraphRAG

```
import argparse
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

import networkx as nx
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field


PROJECT_ROOT = Path(__file__).resolve().parent.parent
DEFAULT_CODEBASE = PROJECT_ROOT / "sample_project"

CODE_EXTENSIONS = {".py", ".ts", ".js", ".java", ".go", ".rs", ".md"}
SKIP_DIRS = {".git", ".venv", "venv", "__pycache__", "node_modules", ".mypy_cache",
".ruff_cache"}


# ---------------------------------------------------------------------------
# Models
# ---------------------------------------------------------------------------

class CodeRelationship(BaseModel):
   subject: str = Field(description="Class, function, module, or file path")
   predicate: str = Field(
       description="Relationship type (e.g., DEFINES, IMPORTS, USES, CALLS,
DEPENDS_ON, INHERITS_FROM)"
   )
   obj: str = Field(description="Target class, function, module, or file path")


class GraphDocument(BaseModel):
   relationships: list[CodeRelationship] = Field(description="All code
relationships extracted from the source file")


class Entities(BaseModel):
   names: list[str] = Field(
       description="Code entities in the query: class names, function names,
modules, file paths"
```

```
    )


# ---------------------------------------------------------------------------
# 1. LOAD
# ---------------------------------------------------------------------------

def load_codebase(root: Path) -> list[tuple[str, str]]:
    root = root.resolve()
    if not root.is_dir():
        raise FileNotFoundError(f"Codebase path does not exist: {root}")
    files = []
    for path in sorted(root.rglob("*")):
        if not path.is_file():
            continue
        if path.suffix.lower() not in CODE_EXTENSIONS:
            continue
        if any(part in SKIP_DIRS for part in path.parts):
            continue
        files.append((str(path.relative_to(root)),
path.read_text(encoding="utf-8")))
    return files


# ---------------------------------------------------------------------------
# 2. EXTRACT RELATIONSHIPS & BUILD GRAPH
# ---------------------------------------------------------------------------

def extract_relationships(relationship_extractor, files: list[tuple[str, str]]) ->
list[CodeRelationship]:
    relationships = []
    for rel_path, content in files:
        result = relationship_extractor.invoke(
            {"messages": [{"role": "user", "content": f"File:
{rel_path}\n\n{content}"}]}
        )
        relationships.extend(result["structured_response"].relationships)
    return relationships


def build_graph(relationships: list[CodeRelationship]) -> nx.DiGraph:
    graph = nx.DiGraph()
    for r in relationships:
        graph.add_edge(r.subject.strip(), r.obj.strip(), relation=r.predicate)
    return graph
```

```
# ---------------------------------------------------------------------------
# 3. RETRIEVE
# ---------------------------------------------------------------------------

def match_nodes(graph: nx.DiGraph, entity: str) -> list[str]:
    needle = entity.strip().lower()
    return [node for node in graph.nodes() if needle in node.lower() or node.lower()
in needle]


def graph_retrieve(graph: nx.DiGraph, entity_extractor, query: str, depth: int = 2)
-> str:
    result = entity_extractor.invoke({"messages": [{"role": "user", "content":
query}]})
    entities = result["structured_response"].names

    relationships = []
    for entity in entities:
        for node in match_nodes(graph, entity):
            neighbourhood = nx.ego_graph(graph, node, radius=depth, undirected=True)
            for source, target, data in neighbourhood.edges(data=True):
                relationships.append(f"{source} -[{data['relation']}]-> {target}")

    if not relationships:
        return "No relevant graph data found."
    return "Knowledge Graph context:\n" + "\n".join(sorted(set(relationships)))



# ---------------------------------------------------------------------------
# 4. MAIN
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="GraphRAG demo — answer questions
about a codebase using a knowledge graph.")
    parser.add_argument("--repo", type=Path, default=DEFAULT_CODEBASE)
    parser.add_argument("--depth", type=int, default=2, help="Graph neighbourhood
radius (default: 2)")
    args = parser.parse_args()


    files = load_codebase(args.repo.resolve())
```

```
    if not files:
        raise SystemExit(f"No source files found under {args.repo}")


    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

    relationship_extractor = create_agent(
        model=llm,
        tools=[],
        response_format=GraphDocument,
        system_prompt=(
            "Extract code relationships from the source file as (subject, predicate,
object) facts.\n"
            "Use ALL_CAPS predicates such as: DEFINES, IMPORTS, USES, CALLS,
DEPENDS_ON, "
            "INHERITS_FROM, IMPLEMENTS, VALIDATES, SENDS_TO, CONFIGURES.\n"
            "Subjects and objects should be class names, function names, module
paths, or file paths.\n"
            "Capture imports, constructor dependencies, method calls, and
cross-module relationships.\n"
            "Be consistent: use the same name for the same class or module across
relationships."
        ),
    )

    entity_extractor = create_agent(
        model=llm,
        tools=[],
        response_format=Entities,
        system_prompt="Extract code-related entities from the user message: class
names, function names, module names, and file paths.",
    )


    qa_agent = create_agent(
        model=llm,
        tools=[],
        system_prompt=(
            "You are a codebase assistant. Answer ONLY from the context provided in
the user message. "
            "Reference specific classes, files, and relationships when possible. "
            "If you cannot answer from context, say so."
        ),
    )
```

```
    print(f"Loaded {len(files)} files — extracting knowledge graph...")
    relationships = extract_relationships(relationship_extractor, files)


    graph = build_graph(relationships)
    print(f"Graph: {graph.number_of_nodes()} nodes, {graph.number_of_edges()}
edges")

    print("\nKNOWLEDGE GRAPH — All extracted relationships:")
    for source, target, data in sorted(graph.edges(data=True)):
        print(f"  {source:35s} -[{data['relation']:20s}]-> {target}")


    print("\nReady. Ask your question. Type 'exit' to quit.")
    while True:
        question = input("\nYou: ").strip()
        if not question or question.lower() in ("exit", "quit"):
            break

        context = graph_retrieve(graph, entity_extractor, question,
depth=args.depth)
        for step in qa_agent.stream(
            {"messages": [{"role": "user", "content":
f"Context:\n{context}\n\nQuestion: {question}"}]},
            stream_mode="values",
        ):
            last_msg = step["messages"][-1]
            if not getattr(last_msg, "tool_calls", None):
                print(f"Agent: {last_msg.content}")
```

RRF - Fusion Algo for Hybrid RAG

Reciprocal Rank Fusion (RRF) is a rank fusion algorithm used to combine results from multiple retrievers (e.g., BM25 + Vector Search) into a single ranked list.

It is one of the most common techniques used in Hybrid Retrieval systems.

Why do we need RRF?

Different retrievers have different strengths:

- Dense retrieval captures semantic meaning.
- Sparse retrieval captures exact keyword matches.
- Other retrievers may use metadata, graphs, etc.

RRF combines their rankings so that no single retriever dominates the final result.

Core Idea

A document should rank higher if:

- It appears high in a retriever's results.
- It appears in multiple retrievers' results.

RRF rewards both rank position and agreement across retrievers.

Formula

RRF score (d) = Σ 1 / (k + rank)

Where:

- d = document/chunk
- rank = position of the document in a retriever's ranking
- k = smoothing constant (typically 60)

The score is summed across all retrievers that returned the document.

Why use ranks instead of scores?

Different retrievers produce scores on different scales.

Examples:

- BM25 scores
- Cosine similarity scores
- Metadata relevance scores

These scores are not directly comparable.

RRF avoids score normalization by using only rank positions.

What is the role of k?

k is a smoothing constant that reduces the impact of small rank differences.

Without k:

Rank 1 = 1/1 = 1.0

Rank 2 = 1/2 = 0.5

Rank 10 = 1/10 = 0.1

Notice that Rank 1 receives a disproportionately large reward and can dominate the final ranking.

With the commonly used value k = 60:

Rank 1 = 1/61 ≈ 0.0164

Rank 2 = 1/62 ≈ 0.0161

Rank 10 = 1/70 ≈ 0.0143

The gap between ranks becomes much smaller:

1/61 = 0.01639

1/62 = 0.01612

As a result:

- Small rank differences matter less.
- A single rank-1 result cannot dominate the ranking.
- Agreement across multiple retrievers becomes more important than tiny rank differences.

This is why RRF works well in Hybrid RAG systems—it prioritizes consensus between retrievers rather than over-rewarding one retriever's top result.

Advantages

- Simple and efficient
- No training required
- Works across different retrieval methods
- No score normalization needed
- Strong performance in hybrid retrieval systems

RRF combines ranked lists by rewarding documents that are ranked highly and consistently across multiple retrievers, making it the standard fusion technique for Hybrid RAG systems.

```
# ---------------------------------------------------------------------------
# rrf_demo.py — How Reciprocal Rank Fusion works, step by step
#
# No LLM, no embeddings. Pure Python so every number is visible.
# ---------------------------------------------------------------------------

# Imagine a user asks: "How does the app enforce task limits per user?"
#
# Two retrievers independently search the codebase and return ranked lists.
# Neither list is perfect on its own — RRF fuses them into one better list.
QUERY = "How does the app enforce task limits per user?"

# Semantic retriever results — understands concepts, misses exact identifiers
SEMANTIC_RESULTS = [
    "billing.py::BillingService",        # rank 1 — talks about limits conceptually
    "task_service.py::create_task",      # rank 2 — mentions task creation flow
    "permissions.py::PermissionPolicy",  # rank 3 — access control concept
    "auth.py::AuthService",              # rank 4 — loosely related
]
# Lexical (BM25) results — exact token matches, misses conceptual docs
LEXICAL_RESULTS = [
    "billing.py::check_task_limit",      # rank 1 — exact words 'task' + 'limit'
    "billing.py::TASK_LIMITS",           # rank 2 — exact token 'TASK_LIMITS'
    "models.py::Task",                   # rank 3 — exact token 'task'
    "task_service.py::create_task",      # rank 4 — exact token 'task'
]
RRF_K = 60   # smoothing constant — prevents rank-1 from dominating too heavily
# ---------------------------------------------------------------------------
# STEP 1 — Show both ranked lists side by side
# ---------------------------------------------------------------------------
```

```
print(f'  Query: "{QUERY}"')

print()

print("--- STEP 1: What each retriever returned on its own ---")

print()

print(f"  {'Rank':<6}  {'Semantic results':<35}  {'Lexical (BM25) results'}")

print(f"  {'-'*6}  {'-'*35}  {'-'*35}")

for i, (s, l) in enumerate(zip(SEMANTIC_RESULTS, LEXICAL_RESULTS), 1):

   print(f"  {i:<6}  {s:<35}  {l}")

print()

print("  Problem:")

print("  • Semantic missed 'check_task_limit' (exact function name)")

print("  • Lexical missed 'BillingService'    (conceptual match, no exact tokens)")

print("  • 'create_task' appears in both — a signal it's probably relevant")

print()

print("  RRF fuses both lists so neither retriever's blind spots dominate.")

# ---------------------------------------------------------------------------

# STEP 2 — Compute the RRF score for each result

# ---------------------------------------------------------------------------

print()

print("--- STEP 2: RRF formula ---")

print()

print("  RRF score  =  Σ  1 / (k + rank)")

print("               (summed over every retriever that returned this chunk)")

print()

print(f"  k = {RRF_K}  ← smoothing constant")

print()

print("  Why 1/(k+rank)?")

print("  • Rank 1  →  1/(60+1) = 0.0164   (high reward)")

print("  • Rank 2  →  1/(60+2) = 0.0161   (slightly less)")

print("  • Rank 4  →  1/(60+4) = 0.0156   (still rewarded, just less)")

print()
```

```
print("  A chunk found by BOTH retrievers gets TWO terms added together,")

print("  which is why agreement across retrievers pushes a chunk to the top.")

# ------------------------------------------------------------------------

# STEP 3 — Calculate scores for every chunk

# ------------------------------------------------------------------------

print()

print("--- STEP 3: Calculating scores ---")

print()

scores = {}

source_map = {}

def add_scores(results, label):

   for rank, chunk in enumerate(results, 1):

       rrf = 1 / (RRF_K + rank)

       scores[chunk] = scores.get(chunk, 0.0) + rrf

       source_map.setdefault(chunk, []).append((label, rank, round(rrf, 6)))

add_scores(SEMANTIC_RESULTS, "semantic")

add_scores(LEXICAL_RESULTS,  "lexical")

print(f"  {'Chunk':<40}  {'Retriever':<10}  {'Rank':<6}  {'1/(k+rank)':<12}
{'Running total'}")

print(f"  {'-'*40}  {'-'*10}  {'-'*6}  {'-'*12}  {'-'*13}")

running = {}

for chunk in list(dict.fromkeys(SEMANTIC_RESULTS + LEXICAL_RESULTS)):  # preserve
order

   for label, rank, rrf in source_map.get(chunk, []):

       running[chunk] = running.get(chunk, 0.0) + rrf

       print(f"  {chunk:<40}  {label:<10}  {rank:<6}  {rrf:<12.6f}
{running[chunk]:.6f}")

# ------------------------------------------------------------------------

# STEP 4 — Final fused ranking

# ------------------------------------------------------------------------

print()

print("--- STEP 4: Final fused ranking ---")

print()
```

```
ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)

print(f"  {'Rank':<6}  {'RRF score':<12}  {'Found by':<25}  Chunk")

print(f"  {'-'*6}  {'-'*12}  {'-'*25}  {'-'*40}")

for final_rank, (chunk, score) in enumerate(ranked, 1):

    retrievers = " + ".join(label for label, _, _ in source_map[chunk])

    star = "  ◀ both agreed!" if len(source_map[chunk]) == 2 else ""

    print(f"  {final_rank:<6}  {score:<12.6f}  {retrievers:<25}  {chunk}{star}")

# ---------------------------------------------------------------------------

# STEP 5 — Key takeaways

# ---------------------------------------------------------------------------

print()

print("--- STEP 5: Why RRF beats picking one retriever ---")

print()

print("  1. Agreement bonus")

print("     'task_service.py::create_task' was rank 2 (semantic) AND rank 4
(lexical).")

print("     RRF added both scores → it climbed higher than either rank alone.")

print()

print("  2. Covers blind spots")

print("     'check_task_limit' — lexical rank 1, absent from semantic.")

print("     'BillingService'   — semantic rank 1, absent from lexical.")

print("     Both appear in the fused list. Neither retriever's miss is final.")

print()

print("  3. No re-training needed")

print("     RRF is score-free — it only uses rank positions.")

print("     You can fuse any two retrievers without calibrating score scales.")

print()

print("  4. The k constant prevents over-rewarding rank 1")

print("     Without k: rank 1 = 1/1 = 1.0,  rank 2 = 1/2 = 0.5  (huge gap)")

print(f"      With  k={RRF_K}: rank 1 = 1/61 = 0.0164, rank 2 = 1/62 = 0.0161
(smooth)")
```

AgenticRAG

Agentic RAG puts an LLM agent in control of retrieval. Instead of a fixed pipeline, the agent reasons about how and what to retrieve - choosing the right retriever(s) based on the query and decomposing complex questions into sub-questions with separate retrieval passes for each.

```
import argparse
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

from langchain_core.documents import Document
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from rank_bm25 import BM25Okapi

from langchain.agents.middleware import ModelCallLimitMiddleware
from langchain.agents.middleware import ToolCallLimitMiddleware

# ------------------------------------------------------------------------
# 1. LOAD
# ------------------------------------------------------------------------

CHUNK_SIZE = 1500

def load_codebase(repo_path: str) -> list:
    docs = []
    for path in Path(repo_path).rglob("*.py"):
        text = path.read_text(encoding="utf-8", errors="ignore")
        docs.append(Document(page_content=text, metadata={"source": str(path)}))
    return docs

# ------------------------------------------------------------------------
# 2. CHUNK
# ------------------------------------------------------------------------

def chunk_code(docs: list) -> list:
    splitter = RecursiveCharacterTextSplitter.from_language(
        language=Language.PYTHON,
        chunk_size=CHUNK_SIZE,
```

```
        chunk_overlap=32,
    )
    return splitter.split_documents(docs)

# ---------------------------------------------------------------------------
# 3. BUILD INDEXES
# ---------------------------------------------------------------------------

def build_vector_store(chunks: list) -> Chroma:
    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    return Chroma.from_documents(chunks, embedding=embeddings)


def build_bm25_index(chunks: list) -> tuple:
    tokenized = [doc.page_content.lower().split() for doc in chunks]
    return BM25Okapi(tokenized), chunks


# ---------------------------------------------------------------------------
# 4. RETRIEVAL TOOL — inner router agent with two sub-tools
#
# Architecture:
#   retrieval_tool (outer @tool)
#       └─ router agent (create_agent)
#               ├─ semantic_retrieval  — dense vector search via Chroma
#               └─ lexical_retrieval   — sparse BM25 keyword search
#
# The router agent receives the user query and decides which sub-tool(s)
# to call. The outer agent sees only retrieval_tool and delegates all
# retrieval decisions to it.
# ---------------------------------------------------------------------------

def build_retrieval_tool(vector_store: Chroma, bm25: BM25Okapi, bm25_docs: list):
    llm = init_chat_model("gpt-4o-mini", model_provider="openai", temperature=0)

    @tool
    def semantic_retrieval(query: str) -> str:
        """Retrieve code chunks using dense vector (semantic) similarity.
        Use for conceptual or intent-based queries about how something works."""
        docs = vector_store.similarity_search(query, k=4)
        print(f"  [semantic_retrieval] '{query}' → {len(docs)} chunks")
        return "\n\n".join(
            f"# {d.metadata.get('source', 'unknown')}\n{d.page_content}" for d in
docs
        )
```

```
    @tool
    def lexical_retrieval(query: str) -> str:
        """Retrieve code chunks using BM25 keyword matching.
        Use for exact identifiers: function names, class names, error strings."""
        tokens = query.lower().split()
        scores = bm25.get_scores(tokens)
        top_indices = sorted(range(len(scores)), key=lambda i: scores[i],
reverse=True)[:4]
        docs = [bm25_docs[i] for i in top_indices]
        print(f"  [lexical_retrieval]  '{query}' → {len(docs)} chunks")
        return "\n\n".join(
            f"# {d.metadata.get('source', 'unknown')}\n{d.page_content}" for d in
docs
        )

    router_agent = create_agent(
        model=llm,
        tools=[semantic_retrieval, lexical_retrieval],
        system_prompt=(
            "You are a retrieval router for a Python codebase. "
            "Given a query, call the right retrieval tool:\n"
            "- semantic_retrieval: conceptual queries ('how does X work?', 'where is
Y handled?')\n"
            "- lexical_retrieval: exact names or strings ('find ClassName', 'where
is func_name called?')\n"
            "Return all retrieved content without summarising it."
        ),
    )

    @tool
    def retrieval_tool(query: str) -> str:
        """Retrieve relevant code from the codebase for any query.
        Internally routes to semantic or lexical retrieval as appropriate."""
        print(f"\n[retrieval_tool] Routing query: '{query}'")
        result = router_agent.invoke(
            {"messages": [{"role": "user", "content": query}]}
        )
        return str(result["messages"][-1].content)

    return retrieval_tool


# ---------------------------------------------------------------------------
# 5. OUTER AGENT
```

```
# ---------------------------------------------------------------------------

def build_agent(retrieval_tool):
    llm = init_chat_model("gpt-4o-mini", model_provider="openai", temperature=0)
    return create_agent(
        model=llm,
        tools=[retrieval_tool],
        system_prompt=(
            "You are a senior engineer. Always use retrieval_tool before answering.
"
            "Reference specific file and function names. "
            "If not found say 'I could not find that in the codebase'."
        ),
        middleware=[
            ModelCallLimitMiddleware(run_limit=5, exit_behavior="end"),
            ToolCallLimitMiddleware(tool_name="retriever_tool", run_limit=2,
exit_behavior="end")
        ]
    )


# ---------------------------------------------------------------------------
# 6. MAIN
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--repo", default=str(Path(__file__).parent.parent /
"sample_project"))
    args = parser.parse_args()
    repo_path = str(Path(args.repo).resolve())

    docs = load_codebase(repo_path)
    chunks = chunk_code(docs)
    print(f"Loaded {len(docs)} files → {len(chunks)} chunks
(chunk_size={CHUNK_SIZE})")

    vector_store = build_vector_store(chunks)
    bm25, bm25_docs = build_bm25_index(chunks)

    retrieval_tool = build_retrieval_tool(vector_store, bm25, bm25_docs)
    agent = build_agent(retrieval_tool)

    print("Ready. Ask your question. Type 'exit' to quit")
    while True:
```

```
    question = input("\nYou: ").strip()
    if not question or question.lower() in ("exit", "quit"):
        break

    for step in agent.stream(
        {"messages": [{"role": "user", "content": question}]},
        stream_mode="values",
    ):
        last_msg = step["messages"][-1]
        if not getattr(last_msg, "tool_calls", None):
            print(f"Agent: {last_msg.content}")
```

Queries to try-

- Lexical Retrieval - BM25 should be chosen by the router
  - Find the AuditLogger class
  - Where is BillingError defined?
  - Where is SlackWebhook used?
  - Show me the PermissionPolicy class

- Semantic Retrieval - vector search should be chosen
  - How does the app handle user authentication?
  - Where does the system send alerts or messages to users?
  - How are background jobs scheduled and executed?
  - How does the app enforce access control?

- Both Retrievers - Mixed queries - router should call both semantic and lexical (depends on prompt)
  - How does billing work and what errors can it raise?
  - How does TaskService create tasks and what models does it use?

HydeRAG

(Hypothetical Document Embeddings)

Instead of embedding the raw question, the LLM first generates a hypothetical answer document. That document is then embedded and used for retrieval.

This works because the hypothetical document is richer and more similar in structure to the real documents in the corpus - leading to better semantic matches.

```
"""
HyDE (Hypothetical Document Embeddings) demo using latest LangChain APIs.

References used:
- HyDE paper: "Precise Zero-Shot Dense Retrieval without Relevance Labels"
(arXiv:2212.10496)
- LangChain HyDE docs: generate hypothetical answer text, then embed that text for
retrieval.

This script intentionally implements HyDE manually (without langchain-classic HyDE
retriever)
so you can understand each step clearly.
"""

from __future__ import annotations

import argparse
import sys
from pathlib import Path

from dotenv import load_dotenv

load_dotenv()

from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.embeddings import init_embeddings
from langchain.tools import tool
from langchain_core.documents import Document
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter

EMBEDDING_MODEL = "text-embedding-3-small"
OPENAI_MODEL = "gpt-4o-mini"
CHUNK_SIZE = 1500
TOP_K = 4
```

```python
# ---------------------------------------------------------------------------
# 1. LOAD
# ---------------------------------------------------------------------------

def load_codebase(repo_path: str) -> list[Document]:
    docs = []
    for path in Path(repo_path).rglob("*.py"):
        text = path.read_text(encoding="utf-8", errors="ignore")
        docs.append(Document(page_content=text, metadata={"source": str(path)}))
    return docs


# ---------------------------------------------------------------------------
# 2. CHUNK
# ---------------------------------------------------------------------------

def chunk_code(docs: list[Document]) -> list[Document]:
    splitter = RecursiveCharacterTextSplitter.from_language(
        language=Language.PYTHON,
        chunk_size=CHUNK_SIZE,
        chunk_overlap=32,
    )
    return splitter.split_documents(docs)

# ---------------------------------------------------------------------------
# 3. EMBED & STORE
# ---------------------------------------------------------------------------

def build_vector_store(chunks: list[Document]) -> InMemoryVectorStore:
    embeddings = init_embeddings(f"openai:{EMBEDDING_MODEL}")
    return InMemoryVectorStore.from_documents(chunks, embeddings)


# ---------------------------------------------------------------------------
# 4. AGENTS
# ---------------------------------------------------------------------------

def build_hyde_generator():
    """
    Agent that writes a hypothetical answer-document from the user question.

    HyDE core idea:
    - Do NOT embed the raw question directly.
    - First generate a plausible answer passage (hypothetical document).
    - Then embed that hypothetical document and retrieve nearest real docs.
```

```
    """
    return create_agent(
        model=init_chat_model(f"openai:{OPENAI_MODEL}", temperature=0),
        tools=[],
        system_prompt=(
            "You generate a single hypothetical code snippet or document for
retrieval.\n"
            "Write a compact, plausible-looking passage (4-7 sentences or a short
code snippet) "
            "that could answer the question about a Python codebase.\n"
            "Do not add bullet points, no disclaimers, and no markdown."
        ),
    )

def build_qa_agent(vector_store: InMemoryVectorStore):
    """
    Final QA agent that answers only from retrieved context via HyDE retrieval.
    """
    hyde_agent = build_hyde_generator()

    @tool(response_format="content_and_artifact")
    def search_codebase(query: str):
        """Retrieve code chunks with HyDE: generate a hypothetical doc, then
retrieve by its embedding."""
        hypo_response = hyde_agent.invoke(
            {"messages": [{"role": "user", "content": query}]}
        )
        hypothetical_doc = str(hypo_response["messages"][-1].content).strip()
        print(f"\n[HyDE] Hypothetical document:\n{hypothetical_doc}\n")
        docs = vector_store.similarity_search(hypothetical_doc, k=TOP_K)

        content = "\n\n".join(
            f"# {doc.metadata.get('source', 'unknown')}\n{doc.page_content}"
            for doc in docs
        )
        artifact = {
            "hypothetical_document": hypothetical_doc,
            "retrieved_docs": [doc.page_content for doc in docs],
        }
        return content, artifact

    qa_agent = create_agent(
        model=init_chat_model(f"openai:{OPENAI_MODEL}", temperature=0),
        tools=[search_codebase],
        system_prompt=(
```

```
            "You are a senior engineer. Always use search_codebase before
answering.\n"
            "Reference specific file and function names.\n"
            "If not found say 'I could not find that in the codebase'."
        ),
    )
    return qa_agent, hyde_agent


# ---------------------------------------------------------------------------
# 5. MAIN
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="HyDE RAG demo over a Python
codebase.")
    parser.add_argument("--repo", default=str(Path(__file__).parent.parent /
"sample_project"))
    args = parser.parse_args()
    repo_path = str(Path(args.repo).resolve())


    docs = load_codebase(repo_path)
    chunks = chunk_code(docs)
    print(f"Loaded {len(docs)} files → {len(chunks)} chunks
(chunk_size={CHUNK_SIZE})")

    vector_store = build_vector_store(chunks)
    qa_agent, hyde_agent = build_qa_agent(vector_store)

    print("Ready. Ask your question. Type 'exit' to quit")
    while True:
        question = input("\nYou: ").strip()
        if not question or question.lower() in ("exit", "quit"):
            break

        for step in qa_agent.stream(
            {"messages": [{"role": "user", "content": question}]},
            stream_mode="values",
        ):
            last_msg = step["messages"][-1]
            if not getattr(last_msg, "tool_calls", None):
                print(f"Agent: {last_msg.content}")
```

Vectorless RAG

The core problem -

Vector RAG retrieves by *similarity* - text that looks like the query. But similarity ≠ relevance.

A query about "payment failures" won't match "Refund Processing Timeline" even though that's the answer. Chunking makes it worse by splitting coherent sections at arbitrary boundaries.

Vectorless RAG retrieves by *reasoning* instead.

PageIndex - builds a tree index of titles + LLM-generated summaries at index time. At query time, an agent reads only summaries, reasons about which nodes are relevant, then fetches only those. Source text is never seen until the agent explicitly requests it.

https://github.com/VectifyAI/PageIndex

```
"""
PageIndex RAG demo — vectorless retrieval via agentic tree navigation.

Based on: https://github.com/VectifyAI/PageIndex

How PageIndex works:
 INDEX TIME  — build a hierarchical tree index (title + summary per node).
               No embeddings. No vector store.

 QUERY TIME  — an agent reasons over the tree using three tools:
                 get_structure()          → full tree (summaries only, no source)
                 get_chunk_content(id)    → actual source of a specific chunk
               The agent drills into relevant files and fetches only the chunks
               it needs. No cosine search.

Tree structure (3 levels):
 directory  (branch) — LLM summary
 └─ file    (branch) — LLM summary
    └─ chunk (leaf)  — first line as summary, source as content

Reference: VectifyAI/PageIndex — "similarity ≠ relevance, retrieval requires
reasoning"
"""

import argparse
from pathlib import Path

from dotenv import load_dotenv
```

```python
load_dotenv()

from langchain.agents import create_agent
from langchain.agents.middleware import ModelCallLimitMiddleware,
ToolCallLimitMiddleware
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langchain_core.documents import Document
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter

LLM_MODEL = "gpt-4o-mini"
CHUNK_SIZE = 1500


# ---------------------------------------------------------------------------
# 1. PAGE INDEX TREE
# ---------------------------------------------------------------------------

def _llm_summary(llm, prompt: str) -> str:
    return llm.invoke(prompt).content.strip()


def _build_chunk_nodes(file_node_id: str, source: str) -> list[dict]:
    # Split the file into chunks using Python-aware separators (class/def boundaries
first).
    # Each chunk becomes a leaf — the agent fetches these by node_id at query time.
    splitter = RecursiveCharacterTextSplitter.from_language(
        language=Language.PYTHON, chunk_size=CHUNK_SIZE, chunk_overlap=32
    )
    chunks = splitter.split_text(source)
    return [
        {
            "title": f"chunk_{i + 1}",
            "node_id": f"{file_node_id}::chunk_{i + 1}",
            "summary": chunk.splitlines()[0][:120],  # first line as a lightweight
preview
            "is_leaf": True,
            "content": chunk,   # actual source — only exposed when agent calls
get_chunk_content
            "nodes": [],
        }
        for i, chunk in enumerate(chunks)
    ]
```

```
def _build_tree(path: Path, base: Path, llm) -> dict | None:
    rel = str(path.relative_to(base)).replace("\\", "/")

    if path.is_file() and path.suffix == ".py":
        source = path.read_text(encoding="utf-8", errors="ignore")
        print(f"  Summarising {rel} ...")

        # LLM reads the file and writes a summary stored in the index.
        # The agent sees this summary (not the source) when browsing
get_structure().

        summary = _llm_summary(
            llm,
            f"In 1-2 sentences, summarise this Python file for a search index.\n"
            f"File: {rel}\n\n```python\n{source[:3000]}\n```",
        )

        # File is a branch node — its children are the source chunks.
        chunks = _build_chunk_nodes(rel, source)
        return {
            "title": path.name,
            "node_id": rel,
            "summary": summary,
            "is_leaf": False,
            "nodes": chunks,
        }

    if path.is_dir():
        # Recurse into subdirectories; skip hidden dirs and __pycache__.
        children = [
            node
            for child in sorted(path.iterdir())
            if not child.name.startswith(".") and child.name != "__pycache__"
            if (node := _build_tree(child, base, llm)) is not None
        ]
        if not children:
            return None
        # Directory summary is derived from the names of its children (no source
needed).
        summary = _llm_summary(
            llm,
            f"In 1 sentence, summarise this Python package for a search index.\n"
            f"Package: {path.name}\nContains: {', '.join(c['title'] for c in
children)}",
```

```
        )
        return {
            "title": path.name,
            "node_id": rel or "root",
            "summary": summary,
            "is_leaf": False,
            "nodes": children,
        }

    return None

def build_page_index(repo_path: str) -> dict:
    # Entry point: walk the repo and build the full tree. One LLM call per file/dir.
    base = Path(repo_path)
    llm = init_chat_model(LLM_MODEL, model_provider="openai", temperature=0)
    root = _build_tree(base, base, llm) or {
        "title": base.name, "node_id": "root", "summary": "Empty project",
        "is_leaf": False, "nodes": [],
    }
    root["node_id"] = "root"
    root["title"] = base.name
    return root

def _count_leaves(node: dict) -> int:
    if node["is_leaf"]:
        return 1
    return sum(_count_leaves(c) for c in node.get("nodes", []))


def _find_node(node: dict, node_id: str) -> dict | None:
    # Depth-first search for a node by its ID.
    if node["node_id"] == node_id:
        return node
    for child in node.get("nodes", []):
        found = _find_node(child, node_id)
        if found:
            return found
    return None

def _tree_to_str(node: dict, indent: int = 0) -> str:
    # Renders the tree showing only node_id + summary — no source code.
    # This is what the agent reads via get_structure() to decide what to fetch.
    prefix = "  " * indent
    icon = "📄" if node["is_leaf"] else "📁"
    lines = [f"{prefix}{icon} [{node['node_id']}] {node['title']}",
```

```
            f"{prefix}   {node['summary']}"]
    for child in node.get("nodes", []):
        lines.append(_tree_to_str(child, indent + 1))
    return "\n".join(lines)
# ---------------------------------------------------------------------------
# 2. PAGEINDEX AGENT
# ---------------------------------------------------------------------------

def build_pageindex_agent(index: dict):
    llm = init_chat_model(LLM_MODEL, model_provider="openai", temperature=0)

    @tool
    def get_structure() -> str:
        """Get the full codebase tree with summaries. Call this first to find
relevant chunks."""
        return _tree_to_str(index)

    @tool
    def get_chunk_content(node_id: str) -> str:
        """Get the source code of a specific chunk by node_id (must be a leaf
node)."""
        node = _find_node(index, node_id)
        if node is None:
            return f"node_id '{node_id}' not found. Call get_structure() for valid
IDs."
        if not node["is_leaf"]:
            children = [c["node_id"] for c in node.get("nodes", [])]
            return f"'{node_id}' is a branch. Children: {children}"
        return node.get("content", "(empty)")

    return create_agent(
        model=llm,
        tools=[get_structure, get_chunk_content],
        system_prompt=(
            "You are a senior engineer using PageIndex to search a codebase.\n"
            "Always call get_structure() first to find relevant chunk node_ids.\n"
            "Then call get_chunk_content(node_id) only for relevant chunks.\n"
            "Answer from retrieved content only. Reference file and function
names.\n"
            "If not found say 'I could not find that in the codebase'."
        ),
        middleware=[
            ModelCallLimitMiddleware(run_limit=8, exit_behavior="end"),
            ToolCallLimitMiddleware(tool_name="get_chunk_content", run_limit=4,
exit_behavior="end"),
```

```
        ],
    )

# ---------------------------------------------------------------------------
# 3. MAIN
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="PageIndex RAG: agentic tree navigation, no vectors."
    )
    parser.add_argument(
        "--repo",
        default=str(Path(__file__).parent.parent / "sample_project"),
    )
    args = parser.parse_args()
    repo_path = str(Path(args.repo).resolve())


    print("Building PageIndex tree ...")
    index = build_page_index(repo_path)
    print(f"PageIndex ready: {_count_leaves(index)} chunks — no vectors, no
embeddings\n")


    print(_tree_to_str(index))

    agent = build_pageindex_agent(index)

    print("\nReady. Ask your question. Type 'exit' to quit")
    while True:
        question = input("\nYou: ").strip()
        if not question or question.lower() in ("exit", "quit"):
            break

        for step in agent.stream(
            {"messages": [{"role": "user", "content": question}]},
            stream_mode="values",
        ):
            last_msg = step["messages"][-1]
            if not getattr(last_msg, "tool_calls", None):
                print(f"Agent: {last_msg.content}")
```

RAG Whitepapers to Explore

- **HyDE (Hypothetical Document Embeddings)** — Query Expansion / Retrieval Improvement RAG
  https://arxiv.org/abs/2212.10496
- **RAPTOR** — Hierarchical / Tree-Based RAG
  https://arxiv.org/abs/2401.18059
- **GraphRAG (Microsoft)** — Graph-Based Retrieval & Reasoning
  https://arxiv.org/abs/2404.16130
- **Self-RAG** — Self-Reflective / Validation RAG
  https://arxiv.org/abs/2310.11511
- **PageIndex (Vectorless RAG)** — Structure-Aware Retrieval Without Embeddings
  https://github.com/VectifyAI/PageIndex